

Greedy Algorithms Study Guide

Greedy Algorithms Study Guide

1. Overview

Greedy algorithms build a solution **step by step**, always choosing the **locally optimal choice** with the hope of reaching a global optimum. They are fast, intuitive, and often outperform DP when applicable.

Greedy works when:

- The problem exhibits **optimal substructure**.
- A **greedy choice property** exists (local optimum → global optimum).

Greedy is common in scheduling, intervals, resource allocation, and optimization problems.

2. How to Identify a Greedy Problem

A problem may be solvable with greedy if:

- You repeatedly choose the “best” next option.
- Sorting helps reveal structure.
- You can prove that making a local optimal choice never harms the final answer.
- DP feels too heavy for the constraints.

Common signals:

- "Minimum number of..."
 - "Maximum number of..."
 - "Earliest finishing..."
 - "Smallest / largest next step..."
-

3. Core Greedy Patterns

3.1 [Interval Scheduling](#)

Choose intervals that **finish earliest**.

Examples:

- Non-overlapping intervals
- Meeting rooms
- Activity selection

Key idea:

sort by end time
choose interval if it doesn't overlap

3.2 [Interval Merging](#)

Used when combining overlapping intervals.

Examples:

- Merge Intervals
- Insert Interval

Key idea:

sort by start time
merge when intervals overlap

3.3 [Minimum Number of Arrows / Points](#)

Same as interval scheduling but reversed perspective.

3.4 [Greedy with Sorting](#)

Used when ordering items reveals optimal choices.

Examples:

- Assign cookies
 - Reconstruct queue by height
 - Task scheduling
-

3.5 [Greedy with Heaps](#)

Used when repeatedly selecting the best or worst element.

Examples:

- Last Stone Weight
 - K largest elements
 - Scheduling with deadlines
-

3.6 [Greedy for Strings](#)

Used when lexicographic order matters.

Examples:

- Remove K digits

- Monotonic stack problems
-

4. Example Problem: Non-overlapping Intervals

Goal: Select the maximum number of non-overlapping intervals.

Solution:

```
def eraseOverlapIntervals(intervals):
    intervals.sort(key=lambda x: x[1])
    count = 0
    end = float('-inf')

    for s, e in intervals:
        if s >= end:
            count += 1
            end = e

    return len(intervals) - count
```

Why greedy works:

- Choosing the interval that ends earliest leaves room for more intervals.
-

5. Example Problem: Jump Game

Goal: Determine if you can reach the last index.

Greedy solution:

```
def canJump(nums):
    reach = 0
    for i, num in enumerate(nums):
        if i > reach:
            return False
        reach = max(reach, i + num)
    return True
```

Why greedy works:

- Always extend the farthest reachable index.
-

6. Common Pitfalls

- Assuming greedy works without proving correctness
- Forgetting to sort
- Choosing the wrong greedy criterion
- Overlooking counterexamples

- Using greedy when DP is required (e.g., knapsack 0/1)
-

7. Practice Roadmap

Beginner

- Assign Cookies
- Lemonade Change
- Jump Game
- Maximum Subarray (Kadane's Algorithm)

Intermediate

- Non-overlapping Intervals
- Merge Intervals
- Partition Labels
- Gas Station

Advanced

- Reorganize String
 - Task Scheduler
 - Remove K Digits
 - IPO (maximize capital)
-

8. Key Takeaways

- Greedy is fast and elegant when applicable.
 - Correctness requires proving the greedy choice is safe.
 - Sorting and heaps are common tools.
 - Many interval problems are greedy.
-

End of document.